Parul® University
Vadodara, Gujarat

NAAC GRADE A++

DEPARTMENT OF
ARTIFICIAL INTELLIGENCE
& MACHINE LEARNING

Java Multithreading

Prof. Ashwani Mishra
Assistant Professor
AIML Department

What Is Multithreading?

- Multithreading allows a single Java program to execute multiple threads of control concurrently.
- Each thread has its own program counter and call stack but shares the process heap memory.
- The JVM maps Java threads to native operating system threads for actual parallel execution.
- Multithreading improves responsiveness in GUI apps and throughput on multi-core processors.
- Incorrect thread coordination can cause race conditions, deadlocks, and unpredictable program behavior.

Figure 7.1 Benefits and Risks of Multithreading

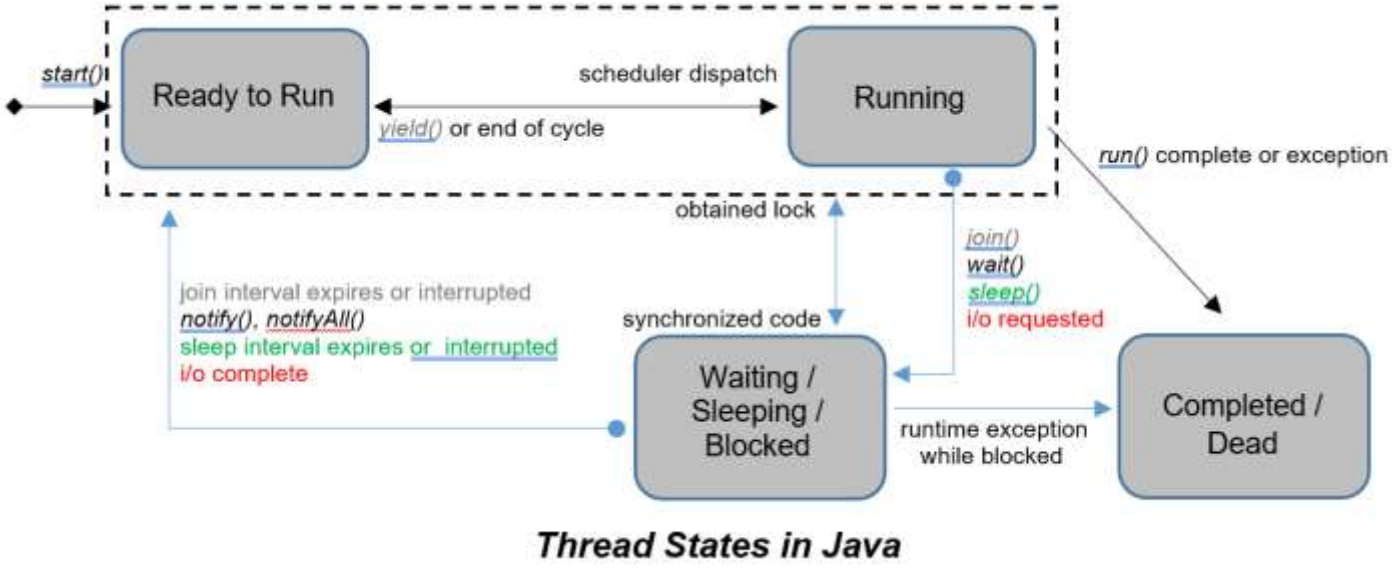


Figure 6.1 Benefits and Risks of Multithreading

Process vs Thread

Feature	Process	Thread
Resource ownership	Owns its own memory and resources	Shares memory and resources with the parent process
Lifetime	Independent lifetime	Dependent on the parent process lifetime
Creation	More expensive to create	Less expensive to create
Context switching	Expensive	Less expensive
Example	Independent projects in different departments	Team members working on a single project

Figure 6.2 Process vs Thread Comparison

Why Programs Use Concurrency

- Responsive user interfaces run long tasks on background threads while the UI thread stays active.
- Server applications handle multiple client requests simultaneously using a thread per request.
- Parallel computation on multi-core CPUs divides work across threads for faster total execution.
- Shared memory between threads enables efficient data exchange without expensive IPC mechanisms.
- Developers must apply synchronization because shared mutable state is vulnerable to race conditions.

Figure 7.3 Concurrency Use Cases in Java Applications

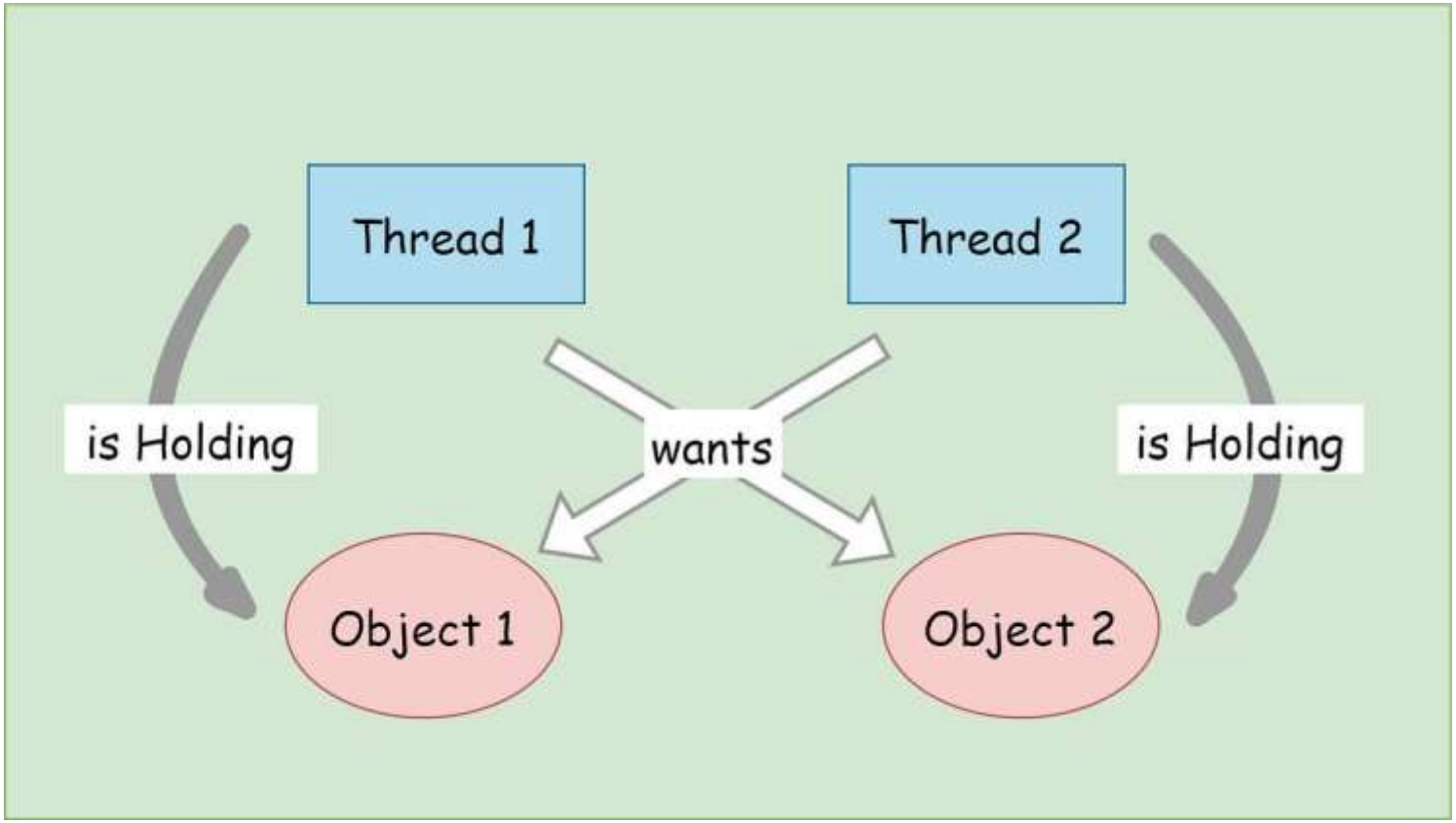


Figure 6.3 Concurrency Use Cases in Java Applications

Thread vs Runnable Approach

Feature	`Thread` Class	`Runnable` Interface
Inheritance	Extends the `Thread` class	Can be implemented by any class
Resource Utilization	Creates a new instance of `Thread` per task	Can be shared among multiple threads
Object Interaction	Can directly access instance variables and methods	Requires an object to interact with (passed as a constructor arg)
Thread Pool Usage	Cannot be used with thread pools	Can be used with thread pools for efficient resource management
Code Reusability	Limited code reusability due to inheritance	Provides better code reusability by implementing interface
Multiple Interfaces	Cannot implement multiple interfaces simultaneously	Allows a class to implement multiple interfaces
Class Extension	Occupies a class inheritance slot	Leaves the class hierarchy unaffected

Figure 6.4 Thread Subclass vs Runnable Interface

Creating a Thread by Extending Thread

- Subclass Thread and override run() to define the code the new thread will execute.
- Calling setName assigns a descriptive name visible in debuggers and thread dump output.
- The start() method creates a new OS thread and schedules run() for asynchronous execution.
- Never call run() directly when concurrency is intended because it runs in the current thread only.
- Extending Thread is discouraged in production because it consumes the single inheritance slot.

Figure 7.5 Thread Creation by Extending Thread

ThreadSubclass.java

```
class MyThread extends Thread {
  @Override
  public void run() {
    for (int i = 1; i <= 5; i++) {
      System.out.println("Thread: " + getName() + " count " + i);
    }
  }
}

public class ThreadSubclass {
  public static void main(String[] args) {
    MyThread t = new MyThread();
    t.setName("Worker-A");
    t.start();  // creates new call stack, invokes run()
  }
}
```

Figure 6.5 Thread Creation by Extending Thread

Creating a Thread with Runnable

- Implement Runnable to separate task logic from the Thread class infrastructure.
- Pass the Runnable instance to the Thread constructor before calling start() on the thread.
- The same Runnable instance can be reused with different Thread objects or thread pool executors.
- This approach follows composition over inheritance and is the standard professional pattern.
- Lambda expressions can implement Runnable concisely: new Thread(() -> { ... }).start().

Figure 7.6 Thread Creation with Runnable Interface

RunnableDemo.java

```
class PrintTask implements Runnable {
  private String name;
  public PrintTask(String name) { this.name = name; }

  @Override
  public void run() {
    for (int i = 1; i <= 5; i++) {
      System.out.println(name + " - step " + i);
    }
  }
}

public class RunnableDemo {
  public static void main(String[] args) {
    Thread t = new Thread(new PrintTask("Worker-B"));
    t.start();
  }
}
```

Figure 6.6 Thread Creation with Runnable Interface

Thread Lifecycle States

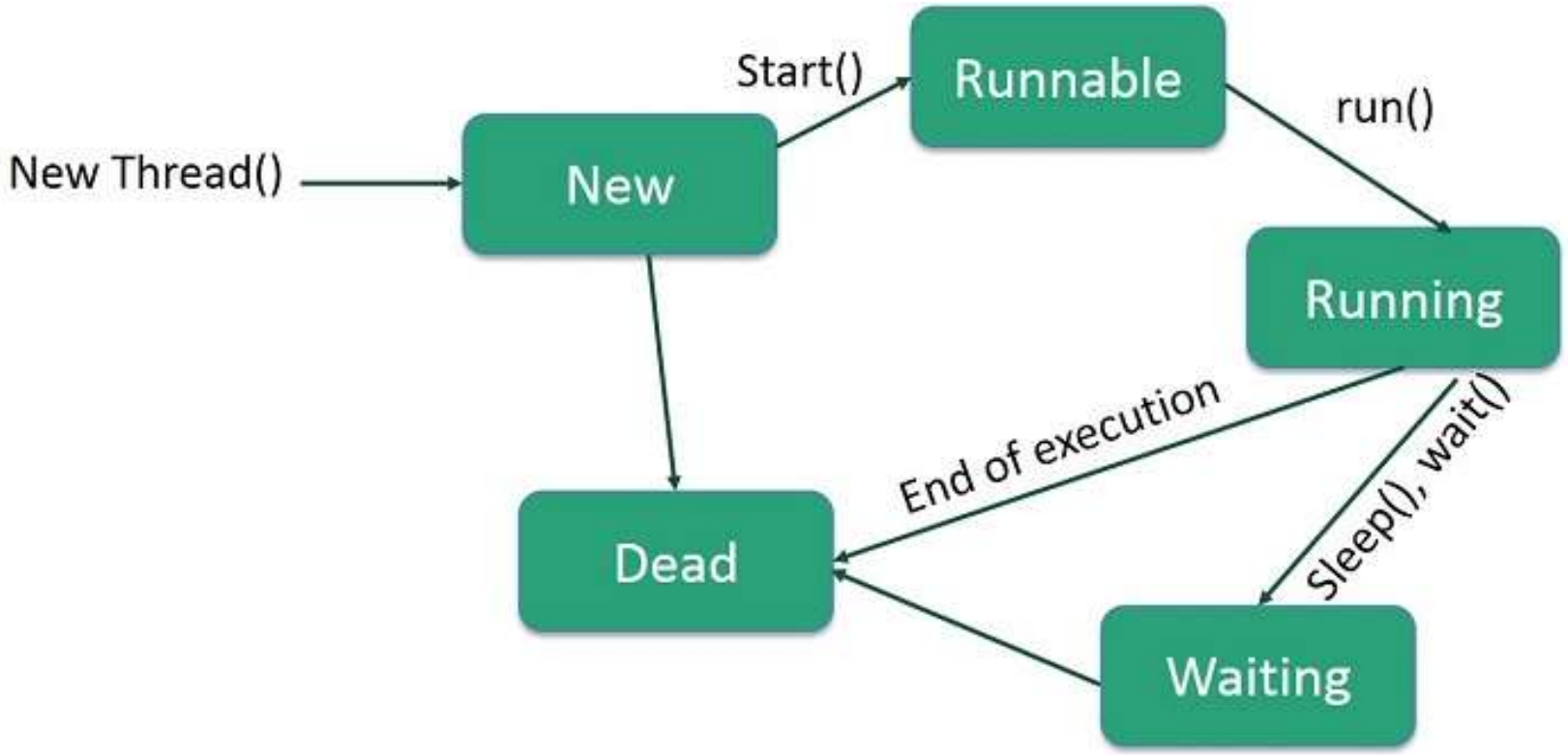


Figure 6.7 Thread Lifecycle State Diagram

start() vs run() Method

Feature	`start()`	`run()`
Thread Creation	Creates a **new thread**	**Does not** create a new thread
Who Executes?	Code in `run()` is executed by the **newly created thread**	Code in `run()` is executed by the **calling thread**
Belongs To	`java.lang.Thread` class	`java.lang.Runnable` interface
Multithreading	Utilizes Java's multithreading capabilities	Does **not** support multithreading
Call Frequency	Can be called **only once** on a thread object	Can be called **multiple times** like a regular method
Use Case	When you want to **start a thread**	When you want to **reuse logic** without threading

Figure 6.8 start() vs run() Behavior

User Threads vs Daemon Threads

- By default every new thread is a user thread that keeps the JVM running until it finishes.
- Calling setDaemon(true) before start() marks a thread as a daemon background helper thread.
- The JVM terminates when all user threads complete, even if daemon threads are still running.
- Daemon threads must not perform critical cleanup because they may be stopped abruptly at shutdown.
- Background log writers and heartbeat monitors are common examples of daemon thread usage.

Figure 7.9 User Threads vs Daemon Threads

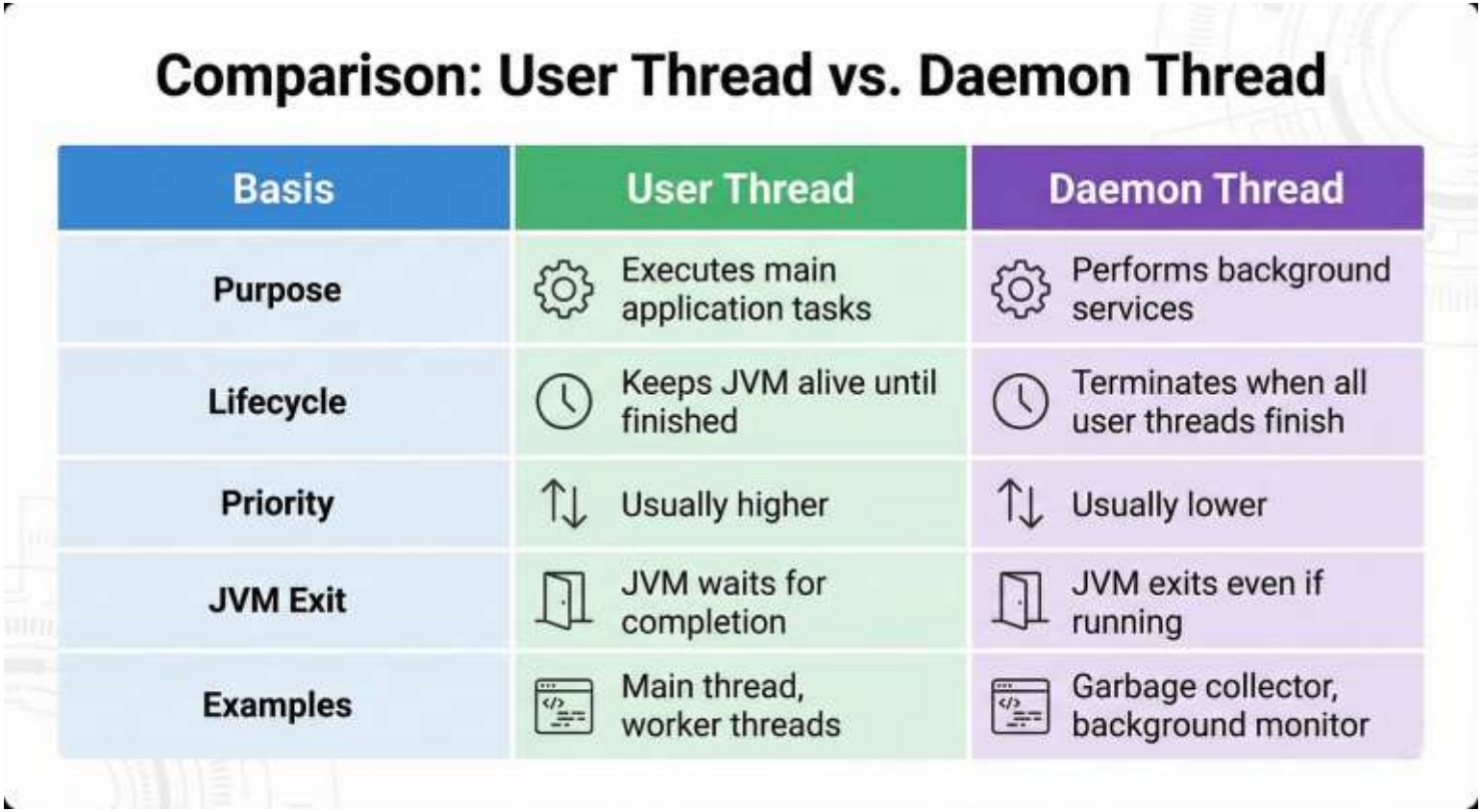


Figure 6.9 User Threads vs Daemon Threads

Thread Methods: sleep, join, and Priority

- Thread.sleep(500) pauses the currently executing thread for the specified milliseconds duration.
- sleep does not release any monitor locks the thread currently holds on synchronized objects.
- The join method blocks the calling thread until the target thread completes its run method.
- setPriority accepts values from Thread.MIN_PRIORITY (1) to Thread.MAX_PRIORITY (10).
- Priority is a hint to the scheduler; actual behavior depends on the underlying operating system.

Figure 7.10 Thread sleep, join, and Priority

ThreadMethods.java

```
Thread worker = new Thread(() -> {
  for (int i = 0; i < 5; i++) {
    System.out.println("Working: " + i);
    Thread.sleep(500);  // pause current thread 500 ms
  }
});

worker.setPriority(Thread.MAX_PRIORITY);  // 1 (MIN) to 10 (MAX)
worker.start();

worker.join();  // main waits until worker finishes
System.out.println("Worker completed. Main continues.");
```

Figure 6.10 Thread sleep, join, and Priority

Important Thread Methods Reference

Important Thread Methods

Method	Description	Notes
start()	Begin thread execution	Calls run() on new thread
run()	Contains thread body logic	Do not call directly for concurrency
sleep(ms)	Pause current thread	Does not release locks held
join()	Wait for thread to die	Main can wait for worker completion
setPriority(int)	Set scheduling priority 1-10	Platform-dependent effect
setDaemon(boolean)	Mark as background thread	Must call before start()

Figure 6.11 Thread Methods Reference Table

Thread Scheduling and Priorities

- The JVM thread scheduler decides which runnable thread executes on an available CPU core.
- Higher-priority threads may preempt lower-priority threads, but this is not guaranteed on all OS platforms.
- Excessive reliance on priority can make programs non-portable across Windows, Linux, and macOS.
- The join method is essential when the main thread must wait for worker threads before proceeding.
- Use thread names and logging to trace concurrent behavior during development and debugging.

Figure 7.12 Thread Scheduling Concepts

Important Thread Methods

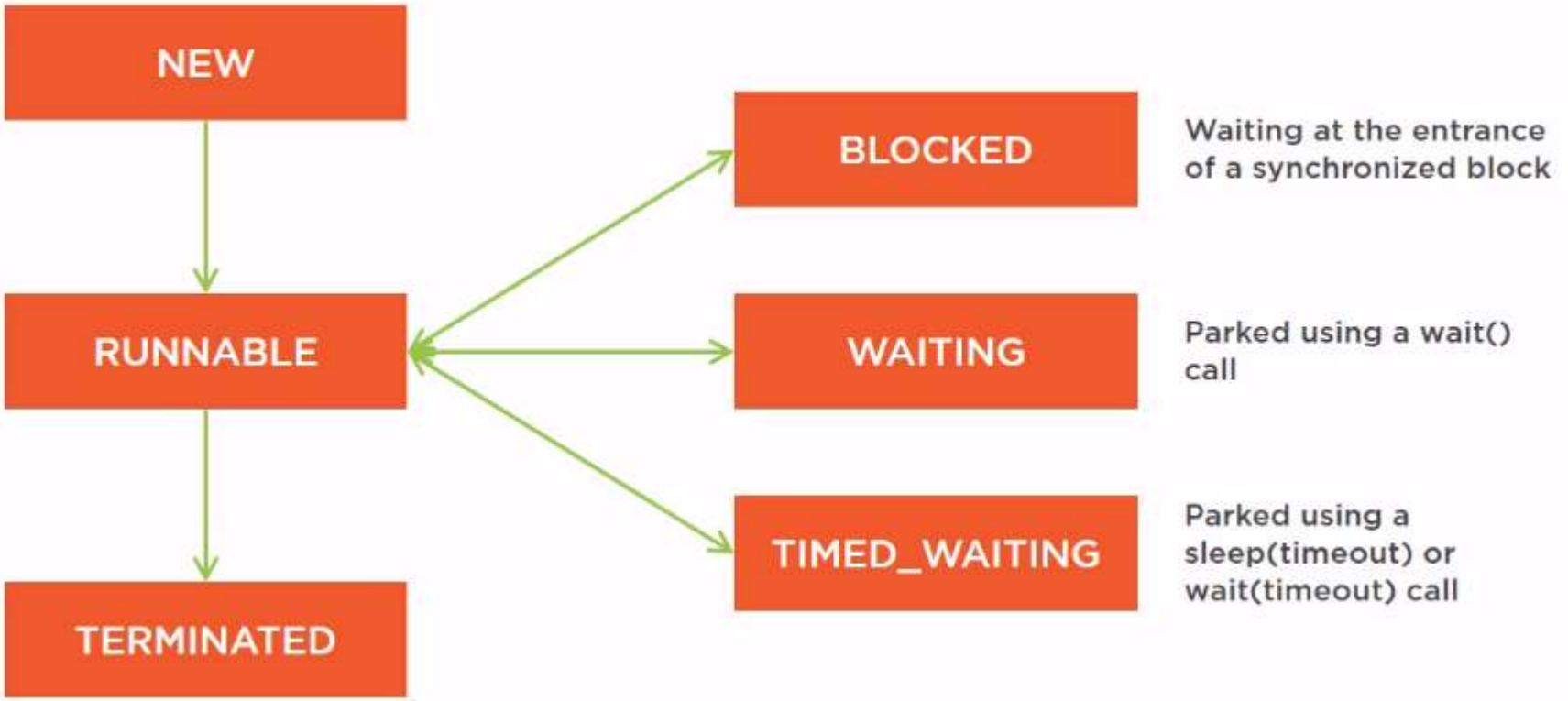
Method	Description	Notes
start()	Begin thread execution	Calls run() on new thread
run()	Contains thread body logic	Do not call directly for concurrency
sleep(ms)	Pause current thread	Does not release locks held
join()	Wait for thread to die	Main can wait for worker completion
setPriority(int)	Set scheduling priority 1-10	Platform-dependent effect
setDaemon(boolean)	Mark as background thread	Must call before start()

Figure 6.12 Thread Scheduling Concepts

Checking Thread State at Runtime

- The Thread.getState() method returns the current lifecycle state such as RUNNABLE or WAITING.
- The isAlive method returns true after start() is called and before the thread terminates completely.
- The interrupt method signals a thread to stop; cooperative code checks isInterrupted inside loops.
- Monitoring thread state helps diagnose deadlocks and unexpected BLOCKED or WAITING conditions.
- Thread dumps in production list every thread state and stack trace for post-mortem analysis.

Figure 7.13 Inspecting Thread State at Runtime



Understanding Race Conditions

- A race condition occurs when two or more threads access shared data and at least one thread writes.
- Without synchronization, interleaved reads and writes produce results that depend on timing luck.
- The lost-update problem happens when two threads read the same value and both write incremented results.
- Race conditions are non-deterministic and may appear only under heavy load or specific timing.
- Identifying shared mutable state is the first step toward applying proper synchronization.

Figure 7.14 Race Condition Lost-Update Problem

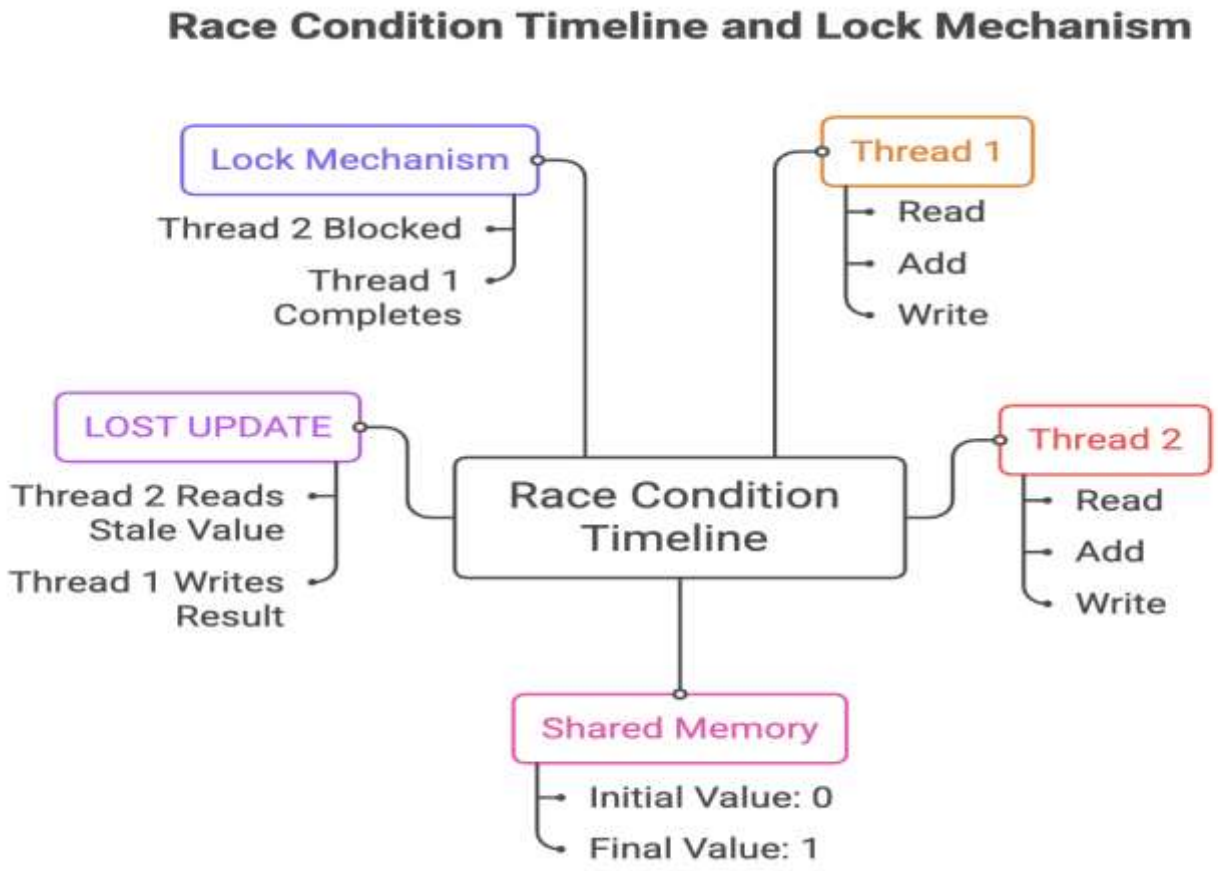


Figure 6.14 Race Condition Lost-Update Problem

The synchronized Keyword

- Declaring increment as synchronized ensures only one thread executes it on a given object at a time.
- The synchronized method acquires the intrinsic lock of this before entering the method body.
- Other threads calling synchronized methods on the same object block until the lock is released.
- A synchronized block on a specific object protects only the critical section inside the braces.
- Proper locking guarantees that count++ executes atomically with respect to other synchronized access.

Figure 7.15 synchronized Method and Block Examples

SynchronizedCounter.java

```
class Counter {
  private int count = 0;

  public synchronized void increment() {
    count++;  // only one thread at a time
  }

  public synchronized int getCount() {
    return count;
  }
}

// Synchronized block on a specific lock object
synchronized (counter) {
  counter.increment();
}
```

Figure 6.15 synchronized Method and Block Examples

synchronized Method vs synchronized Block

Aspect	Synchronized Method	Synchronized Block
Locking Scope	Locks the entire method.	Locks only the critical section of code.
Granularity	Coarse-grained (locks entire method).	Fine-grained (locks specific block).
Lock Target	Locks on the object (for instance methods) or class (for static methods).	Locks on any object you specify.
Performance	Can be less efficient if the entire method is locked, especially if the method has non-critical code.	More efficient as it minimizes the locked region, allowing other threads to access the non-critical parts of the method.
Flexibility	Less flexible, as it synchronizes the whole method.	More flexible, as you can synchronize only the part of the method that needs to be synchronized.

Synchronization Best Practices

- Keep synchronized blocks as short as possible to maximize concurrency among waiting threads.
- Always synchronize on the same lock object that guards the shared mutable data being protected.
- Avoid synchronizing on publicly accessible objects that external code might also lock unexpectedly.
- Prefer java.util.concurrent utilities such as AtomicInteger for simple atomic counter operations.
- Test multithreaded code under load because race conditions often hide in single-threaded test runs.

Figure 7.17 Synchronization Best Practices

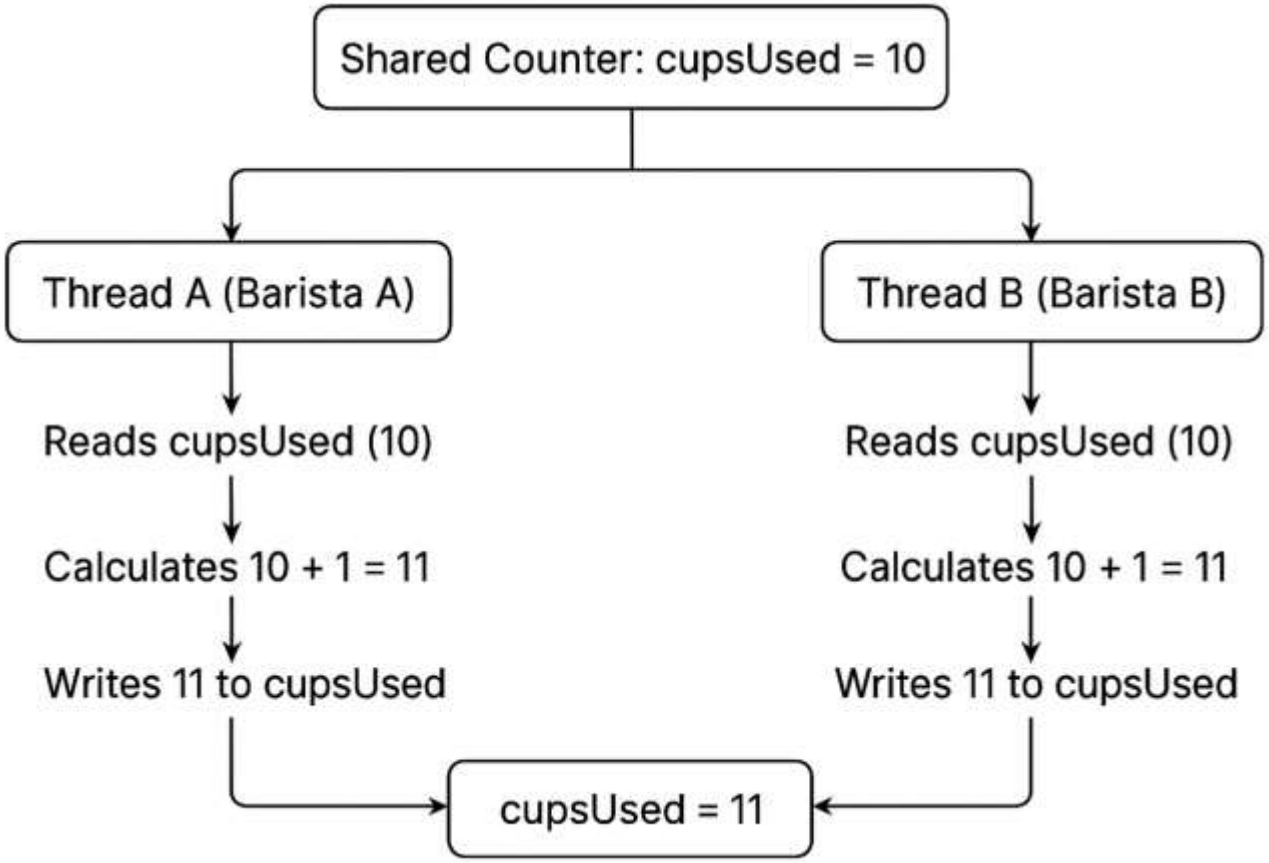


Figure 6.17 Synchronization Best Practices

References

1. Herbert Schildt, Java: The Complete Reference
Cay S. Horstmann, Core Java, Volume I - Fundamentals
Kathy Sierra and Bert Bates, Head First Java

Parul® University | NAAC GRADE A++

https://paruluniversity.ac.in/